

Skill Exchange Platform

A platform to list, discover, and exchange skills between users

Group Number: 1

Submitted by

Name	Register Number
Kanishma	MS254220
Thejeshwini	MS254248
Krishna Priya	MS254221
Asaph	MS254206

27 September 2026

Aim

The aim of this project is to design and develop a full-stack web application — the Skill Exchange Platform — that allows users to list skills they can teach and skills they wish to learn, discover other members with complementary skills, send and manage exchange requests, communicate through in-app messaging, and rate one another after a completed exchange, with no monetary transaction involved.

Objectives

- Build a secure user authentication system using password hashing and token-based sessions.
- Allow users to maintain a profile listing skills offered and skills wanted.
- Enable searching and browsing of other members by skill.
- Implement a request-based matching system for proposing skill exchanges.
- Provide in-app messaging between users once a request is accepted.
- Implement a rating system so users can review each other after an exchange.
- Apply full-stack development practices: REST API design, database modelling, and client-server communication.
- Use Git and GitHub for version-controlled, collaborative team development.

Scope

The system is built as a web application accessible via a browser, using a Node.js/Express backend, a MongoDB Atlas cloud database, and a plain HTML, CSS, and JavaScript frontend. It is intended as an academic full-stack web (FSW) project demonstrating the complete request-response cycle across frontend, backend, and database layers.

1. Introduction

Skill-sharing communities have grown as an alternative to paid learning platforms, letting people trade knowledge directly with one another. The Skill Exchange Platform addresses this need by giving users a simple way to advertise what they can teach, state what they want to learn, and connect with a compatible match — all without any payment involved.

The project was built collaboratively by a team of four, following a full-stack development workflow: planning the data model, building a REST API, developing a matching browser client, and testing the complete flow from account creation through to rating an exchange.

2. Technology Stack

2.1 Frontend

- HTML5, CSS3 (custom design system — forest green and amber palette, Fraunces and Inter typefaces)
- Vanilla JavaScript (ES6+), using the Fetch API for all client-server communication

2.2 Backend

- Node.js with the Express.js framework
- RESTful API design under the /api namespace
- JSON Web Tokens (JWT) for stateless authentication
- bcrypt.js for password hashing

2.3 Database

- MongoDB Atlas (cloud-hosted, free-tier cluster)
- Mongoose as the Object Data Modelling (ODM) library

2.4 Tools

- Visual Studio Code as the IDE
- Git for version control, GitHub for the shared remote repository
- npm for dependency and script management

3. System Architecture

The application follows a client-server architecture. The Express server both serves the static frontend files and exposes a REST API. The frontend communicates with the backend exclusively through `fetch()` calls to JSON endpoints, authenticated using a JWT bearer token stored in the browser's `localStorage`. The backend validates each request, applies business logic, and reads from or writes to the MongoDB Atlas database via Mongoose models.

```
Browser (HTML/CSS/JS)
  | fetch() + JWT Bearer token
  v
Express.js server --serves--> frontend/ (static files)
  | /api routes (auth, users, requests, messages, ratings)
  v
Mongoose models <--> MongoDB Atlas (cloud database)
```

3.1 Database Schema (Collections)

- User — name, email, hashed password, bio, skillsTeach[], skillsWant[], ratingAvg, ratingCount
- Request — fromUser, toUser, skillOffered, skillWanted, message, status
- Message — requestId, sender, text, createdAt
- Rating — requestId, fromUser, toUser, stars, comment

4. Features Implemented

Feature	Description
User Authentication	Signup and login with hashed passwords (bcrypt) and JWT-based sessions.
Profile Management	Users set a bio, the skills they can teach, and the skills they want to learn. Includes a live bio character counter.
Browse & Search	Search other members by skill keyword. Results show a live count and are sorted by average rating.
Exchange Requests	Send a request offering one skill in exchange for another; recipient can accept or reject.
Messaging	Once a request is accepted, both users can chat in a conversation thread with timestamps, auto-refreshing every few seconds.
Ratings	After a completed exchange, either user can leave a 1-5 star rating (via a clickable star picker) and a comment, updating the other user's average rating.

5. Individual Contributions

The team divided the application into ownership areas, with each member responsible for building, testing, committing, and pushing their part to the shared GitHub repository under their own account, so that individual contributions are reflected in the commit history.

Member	Module Owned	Contribution
Kanishma	Project setup (Node.js, Git, MongoDB Atlas, GitHub repo), Auth & Profile module	Built live bio character counter on the profile page; added a clearer guidance message when signing up with an email that is already registered.
Thejeshwini	Browse & Requests module	Added a live result count to the Browse page and implemented sorting of members by average rating.
Krishna Priya	Messaging & Ratings module	Added timestamps to chat messages and replaced the numeric rating input with a clickable 5-star rating picker.
Asaph	Testing & Quality	End-to-end testing of the signup → browse → request → message → rate flow across test accounts.

6. Code Excerpts

6.1 Backend — Signup Route (backend/routes/auth.js)

Handles new user registration with input validation, duplicate-email checking, and password hashing before saving to the database.

```
router.post('/signup', async (req, res) => {
  const { name, email, password } = req.body;
  const existing = await User.findOne({ email });
  if (existing) return res.status(409).json({
    error: 'An account with this email already exists.' });

  const hashedPassword = await bcrypt.hash(password, 10);
  const user = await User.create({ name, email, password: hashedPassword });

  const token = jwt.sign({ userId: user._id },
    process.env.JWT_SECRET, { expiresIn: '7d' });
  res.status(201).json({ token, user });
});
```

6.2 Backend — JWT Authentication Middleware (backend/middleware/auth.js)

Protects private routes by verifying the JWT token sent in the Authorization header before allowing the request to proceed.

```
module.exports = function auth(req, res, next) {
  const header = req.headers.authorization;
  if (!header || !header.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'No token provided.' });
  }
  const token = header.split(' ')[1];
  const decoded = jwt.verify(token, process.env.JWT_SECRET);
  req.userId = decoded.userId;
  next();
};
```

6.3 Frontend — Live Bio Character Counter (frontend/js/dashboard.js)

Updates a character counter beneath the bio field as the user types, reflecting the 300-character limit.

```
document.getElementById('bio').addEventListener('input', (e) => {
  document.getElementById('bioCount').textContent = e.target.value.length;
});
```

6.4 Frontend — Clickable Star Rating Picker (frontend/js/requests.js)

Lets a user click on one of five star icons to select a rating from 1 to 5, highlighting all stars up to the selected value.

```
starItems.forEach(star => {
```

```
star.addEventListener('click', () => {  
  const value = Number(star.getAttribute('data-value'));  
  starsInput.value = value;  
  starItems.forEach(s => {  
    const sValue = Number(s.getAttribute('data-value'));  
    s.style.color = sValue <= value ? '#E8A23D' : '#DDE3DD';  
  });  
});  
});
```

7. Output Screenshots

7.1 Landing Page

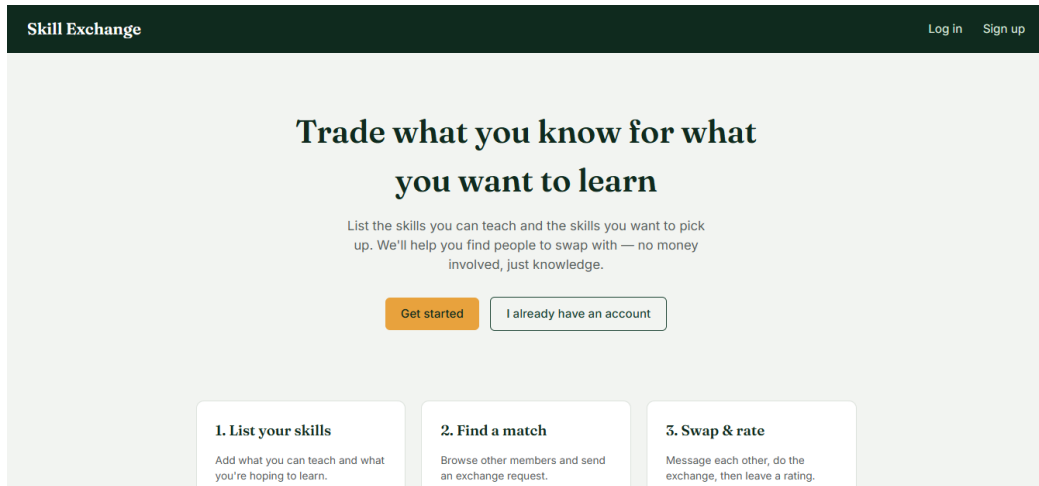


Figure 1: Landing page introducing the platform and its three-step flow.

7.2 Server Running and Connected to MongoDB Atlas

```
PS C:\Users\hp\Desktop\lab exams\Skill-Exchange-Platform> npm start

> skill-exchange-platform@1.0.0 start
> node backend/server.js

Connected to MongoDB
Server running at http://localhost:3000
█
```

Figure 2: Express server successfully connected to the MongoDB Atlas database.

7.3 Profile Page — Live Bio Character Counter

The screenshot shows a portion of a web form. At the top, there is a horizontal line with a small vertical segment at the left end, indicating a text input field. Below this line, the text '30/300 characters' is displayed in a gray font. Further down, the label 'Skills I can teach (comma separated)' is shown above a text input field containing the text 'photoshop, DBMS'. Below that, the label 'Skills I want to learn (comma separated)' is shown above another empty text input field.

Figure 3: The bio field showing a live character count as the user types.

7.4 Signup Page — Duplicate Email Guidance

Create your account

An account with this email already exists.

Full name

theju

Email

kanishma0509@gmail.com

Password

.....

At least 6 characters.

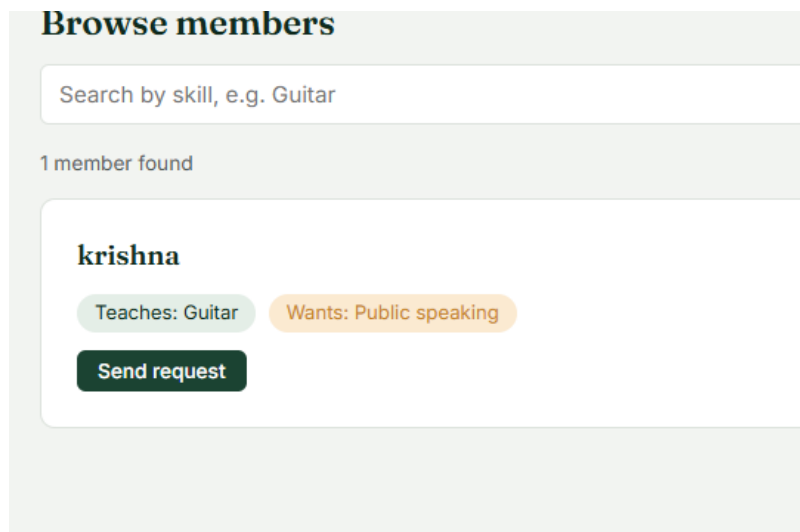
Create account

Already have an account? [Log in](#)

Tip: if this email is already registered, try logging in instead.

Figure 4: Clear error and guidance shown when signing up with an already-registered email.

7.5 Browse Page — Search Results with Count



Browse members

Search by skill, e.g. Guitar

1 member found

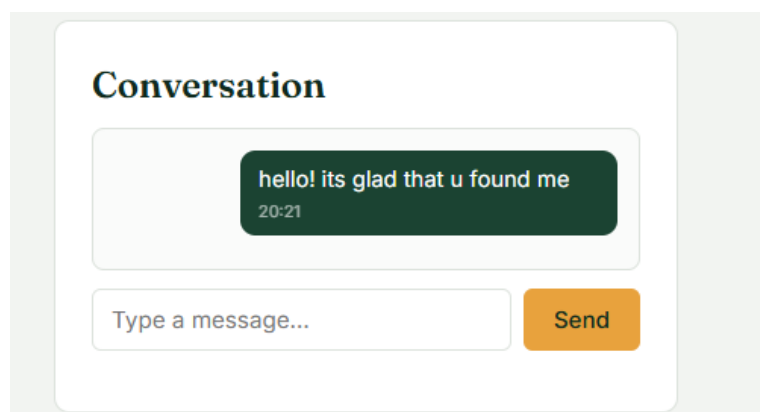
krishna

Teaches: Guitar Wants: Public speaking

Send request

Figure 5: Browse page showing the live result count and a matched member's skills.

7.6 Messaging — Conversation with Timestamps



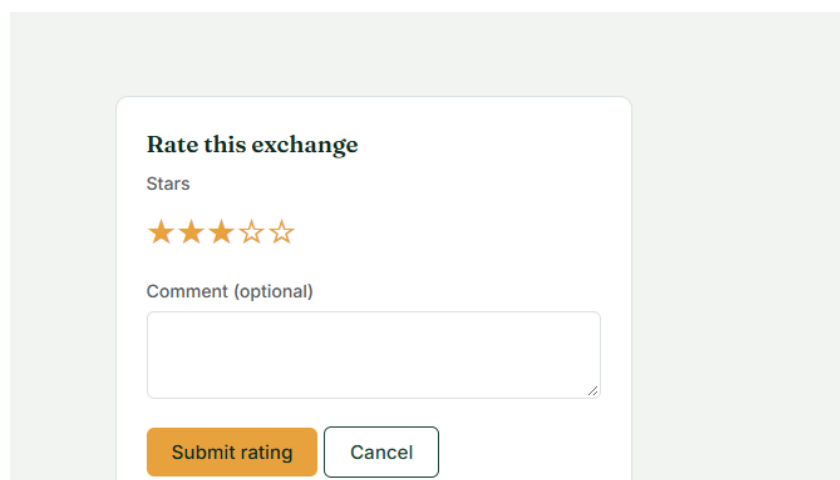
Conversation

hello! its glad that u found me
20:21

Type a message... Send

Figure 6: In-app chat between two matched users, with message timestamps.

7.7 Ratings — Clickable Star Picker



Rate this exchange

Stars

★★★★☆

Comment (optional)

Submit rating Cancel

Figure 7: Rating form using a clickable five-star picker instead of a plain number field.

7.8 GitHub — Team Commit History

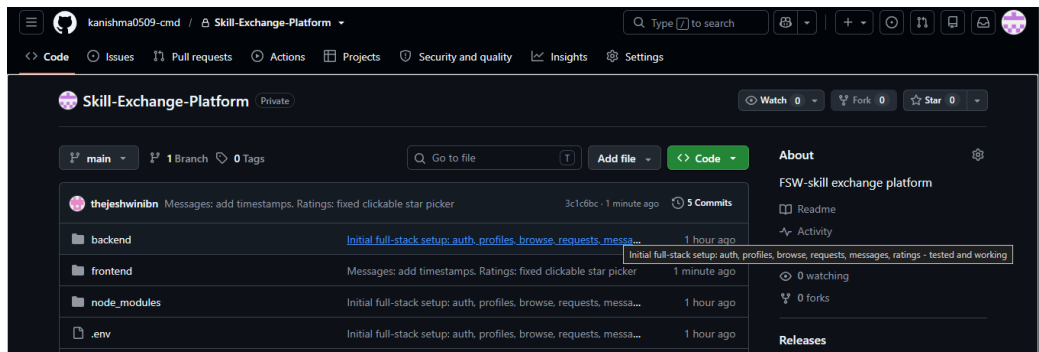


Figure 8: Commit history on the shared GitHub repository showing contributions from multiple team members.

8. Conclusion

The Skill Exchange Platform was successfully designed, built, and tested as a complete full-stack web application. The project demonstrates the full request-response cycle across a JavaScript-based frontend, a Node.js/Express backend, and a MongoDB Atlas cloud database, along with practical use of authentication, REST APIs, and Git-based team collaboration.

All core objectives were met: users can create accounts, list the skills they teach and want to learn, browse and search other members, send and respond to exchange requests, message matched users, and rate one another after an exchange. The team divided work across authentication/profile, browse/requests, and messaging/ratings, with each member's contributions committed and pushed individually to a shared, private GitHub repository.

8.1 Learning Outcomes

- Practical experience with the full stack: frontend, backend, REST API, and database.
- Hands-on use of JWT authentication and password hashing for secure user accounts.
- Working with a cloud-hosted database (MongoDB Atlas) and environment-based configuration.
- Collaborative software development using Git and GitHub, including resolving a real security issue (an accidentally committed `.env` file) by rotating credentials and correcting `.gitignore`.

8.2 Future Enhancements

- Real-time messaging using WebSockets instead of periodic polling.
- Profile photo uploads.
- Email notifications for new requests and messages.
- Deployment to a public hosting service (e.g. Render, Railway) for live access.